

Assignment 2

Objective: use a bottom-up methodology to practice functions, call-by-reference parameters and I/O Stream in C++.

In this assignment, you need to simulate a Blackjack game.

What is Blackjack?

Blackjack, also known as **21**, is one of the most popular and widely played casino card games in the world. It combines elements of luck and strategy, making it appealing to both casual players and serious gamblers. The primary objective of Blackjack is to have a hand value **closer to 21** than the dealer's hand without exceeding (or "busting") the total.

Basic Rules and Gameplay

1. The Deck:

- Blackjack is typically played with one or more standard decks of 52 cards.
- Each card has a point value:
 - **Number cards (2–10):** Worth their face value.
 - **Face cards (Jack, Queen, King):** Worth **10 points** each.
 - **Aces (A):** Can be worth **1 or 11 points**, depending on which value benefits the hand more.

2. Players and Dealer:

- The game involves **one or more players** competing against the **dealer** (who represents the house).
- Each player plays individually against the dealer, not against other players.

3. Starting the Game:

- **Betting:** Players place their bets before any cards are dealt.
- **Dealing Cards:**
 - Each player is dealt **two cards** face up.
 - The dealer receives **two cards**: typically, one **face up** (the "upcard") and one **face down** (the "hole card").

4. Player's Turn:

- Players decide how to play their hands based on the total point value and the dealer's upcard.
- **Options include:**
 - **Hit:** Take another card to increase the hand's total value.
 - **Stand:** Keep the current hand and end the turn.
 - **Double Down:** Double the initial bet and receive exactly one more card.
 - **Split:** If the first two cards have the same value, they can be split into two separate hands, each with its own bet.
 - **Surrender:** Forfeit half the bet and end the hand immediately (availability varies by casino rules).

5. Dealer's Turn:

- After all players have completed their actions, the dealer reveals the hole card.
- The dealer must follow strict rules:
 - **Hit** until the hand's total value is **17 or higher**.
 - **Stand** once the total is **17 or higher**.
 - Some casinos require the dealer to **hit on a "soft 17"** (a hand totaling 17 that includes an Ace counted as 11).

6. Determining the Winner:

- **Blackjack:** If a player's initial two cards total **21** (an Ace and a 10-point card), it's called a "Blackjack" and typically pays out **3:2**.
- **Bust:** If a player's or dealer's hand exceeds **21**, they bust and lose the round.
- **Comparison:**
 - If neither bust, the hand closest to **21** wins.
 - If both the player and dealer have the same total, it's a **push** (a tie), and the player's bet is returned.

Requirements for this assignment:

Write a simplified dice version of blackjack. Instead of selecting from a deck of cards, a 14-sided die is rolled to represent a card. The rules of the game for one player in one round, as played against the computer (House), are as follows:

- 1) The player has an initial amount of money in hand. (for example, 500 dollars)
- 2) A roll randomly selects from the values 2-10, or Jack, Queen, King, or Ace. An Ace always has the value of 11. Jack, Queen, and King all have the value of 10. In other words, with equal probability, a roll selects one of 2, 3, 4, 5, 6, 7, 8, 9, 10, 10, 10, 10, 11.
- 3) The human player enters a wager. The wager must be positive and must not exceed the amount of money possessed by the player. The amount of wager should be deducted from the money in hand.
- 4) The player is given two rolls. The house (computer) is given one roll. The player is shown the value of the house's roll. If the total of the rolls for player is 22 (which means two Aces), your program can treat it as blackjack. In real life, the player will split, might double down based on the face card of the dealer.
- 5) If the total of the rolls is 22 or greater then the player "busts", loses the wager, and the round is over.
- 6) If the total of the rolls is 21, then player will "stand", if the total of the rolls is less than 21, then player has the option to "hit" or "stand". If the player opts to "hit" then the player is given another roll. Return to step 5. If the player opts to "stand", then continue to step 7.
- 7) It's now the house's turn, the house is given an additional roll and the value are summed. If the total is less than 17, then the house must "hit". If the house has a total of 22 or greater then the house "busts". The player wins the wager, and the round is over. The wager $\times 2$ (wager deducted in step 3 + the win) should be added back to the balance of the player's balance. The house must continue to hit until it either has a value between 17 and 21 or "busts".
- 8) If the house did not "bust" then it's value is compared to the player's total. If the totals are the same, then it's a "push" (tie), the player does not win or lose. If the house's total is greater than the player's, then the player loses the wager. If the player's total is greater than the house's, then the player wins the wager. Remember to update the balance of player's money.

Your program should repeat a round until the player decides to quit or runs out of money.

Bottom-up Development Methodology:

Write your program using the bottom-up development methodology. When you write your code, write functions before writing higher-level code to implement the game logic, use driver to test your functions. You can start with the following functions then test them in main().

int rollDice()	Rolls 2-14, which will randomly select 2-10, Jack (11), Queen(12), King(13), or Ace(14), then return the values it represent.
----------------	---

Then try to implement other functions, for example,

```
void playerTurn(int money, int &wager, bool &bust, int &total, int &houseroll)
```

in which:

money represents the money player owns;

wager represents the amount of money wagered by the player;

bust represents whether the player is busted or not;

total represents the sum of the rolls by the player;

houseroll represents the single roll made by the house as explained in step 4 above.

Test each of the functions to ensure that they are working properly. Finally write the logic for the final game.

Output

The game should output the following information **to both the screen and a textfile**:

- The cards(numbers) in the player's hand and their total value.
- Whether the player chooses to hit or stand.
- The dealer's final hand and total value.
- Who wins the game.

Grading Criteria:

- Correct implementation of game rules and logic and Proper handling of edge cases (3 points)
- Code readability, structure, and comments (1 point)
- Proper user input handling/validation and output formatting (1 point)

Submission:

Submit the C++ source file (.cpp) and the screenshot & text file of your program output on Moodle. Test your program thoroughly and ensure that your code compiles and runs correctly.